30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 22

Build System & Gradle — Part 2: Dependencies, Variants, R8 & Builds at Scale



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22

Why dependencies and variants matter in 2026

Day 21 made the build fast and tidy. Part 2 is about keeping it **correct at scale** — the class of bugs that pass in debug and crash in release.

- The dependency graph decides what recompiles: `api` everywhere turns one change into a rebuild of half the app
- Gradle resolves **one version** per module on each classpath — often not the one you wrote
- R8 runs on release builds only; a missing keep rule ships a crash. AGP 9 made keep rules stricter
- AGP 9 hides the old DSL types and the legacy variant API by default — upgrades need a strategy
- Custom tasks must be lazy and cacheable, or they undo everything from Part 1

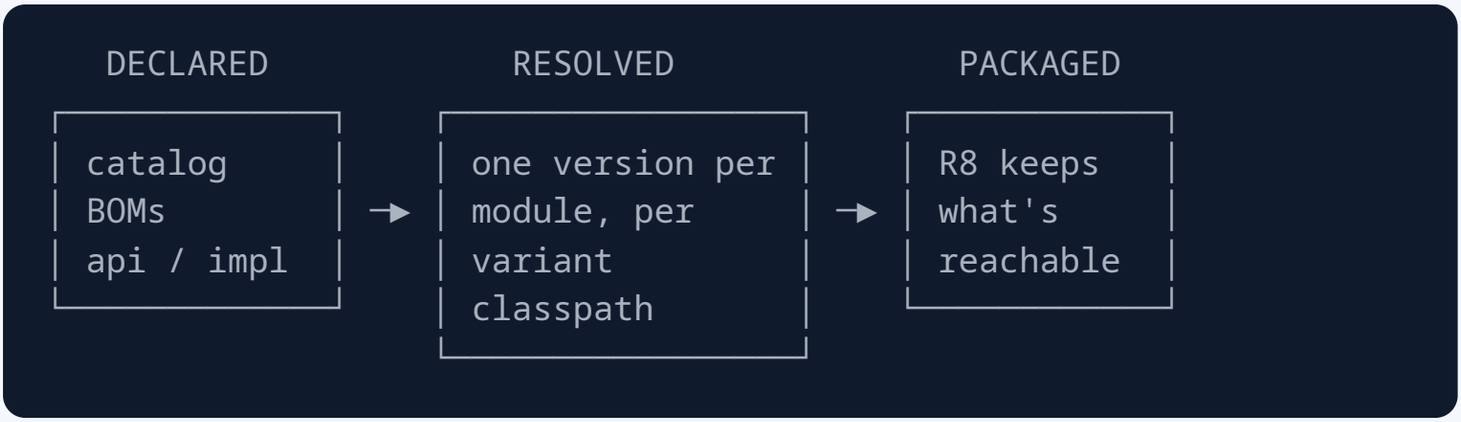
Level	Questions
Junior	`api` vs `implementation` · Build types & flavors · Per-environment config
Mid	Version conflicts · BOMs & platforms · Constraints & rich versions · Lazy custom tasks
Senior	R8 keep rules · Measuring build performance · Remote build cache
Architect	AGP upgrade strategy · Dependency governance · Variant API at scale



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22

Mental model: declared → resolved → packaged



- **Declared** is a request: catalog versions, BOMs and constraints all feed into resolution
- **Resolved** is the truth: one version per module on each classpath (`debugRuntimeClasspath`, `releaseRuntimeClasspath` …) — by default the highest requested
- **Packaged**: R8 removes whatever it can't prove is used; reflection needs keep rules

DEBUG THE RESOLVED GRAPH, NOT THE BUILD FILE

`./gradlew :app:dependencies` shows what actually ships. Your build file only shows what you asked for.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22

Junior

api vs implementation · Build types & flavors · Per-environment config



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. `api` vs `implementation`

Configuration	Your compile classpath	Consumers' compile classpath	Runtime (in the APK)
`implementation`	yes	—	yes
`api`	yes	yes	yes
`compileOnly`	yes	—	—
`runtimeOnly`	—	—	yes

```
// :core:network — Retrofit types appear in its public API
dependencies {
    api(libs.retrofit)                 // consumers see it
    implementation(libs.okhttp.logging) // internal detail
}
```

- `implementation` keeps a dependency off consumers' compile classpath — when it changes, they don't recompile
- Use `api` only when the dependency's types appear in **your** public API: parameters, return types, supertypes, public properties



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"`implementation` keeps the library out of the app." — No. It's still on the runtime classpath and in the APK. It only controls who can **compile** against it.

INTERVIEW TRAP

"Use `api` everywhere so nothing breaks." — Then every module sees everything: builds recompile far more than needed, and modules start depending on each other's internals by accident.

FOLLOW-UP

"How do you find misdeclared dependencies?" — The Dependency Analysis Gradle Plugin (`./gradlew buildHealth`) reports unused dependencies, `api` that should be `implementation`, and transitive dependencies you use without declaring.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Build types, product flavors and variants

```
android {
    flavorDimensions += listOf("tier", "env")
    productFlavors {
        create("free") { dimension = "tier" }
        create("paid") { dimension = "tier" }
        create("dev") {
            dimension = "env"
            applicationIdSuffix = ".dev"
        }
        create("prod") { dimension = "env" }
    }
}
```

- **Build type** = *how* it's built: debuggable, R8, signing (`debug` , `release` , maybe `benchmark`)
- **Product flavor** = *what* is built: features, endpoints, branding — grouped in dimensions
- **Variant** = one flavor per dimension × a build type: 2 × 2 × 2 = **8** variants, `freeDevDebug` … `paidProdRelease`
- Source sets merge; the most specific wins: variant → build type → flavor → `main`



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. The release build type

```
buildTypes.getByName("release") {
    isMinifyEnabled = true   // default: false
    isShrinkResources = true
    proguardFiles(
        getDefaultProguardFile("proguard-android-optimize.txt"),
        "proguard-rules.pro",
    )
}
```

INTERVIEW TRAP

"Release builds are obfuscated with R8 by default." — No: `isMinifyEnabled` is `false` by default, even for `release`. You turn R8 on — and then test the minified build. (Last year's version of this challenge claimed otherwise.)

INTERVIEW TRAP

"The debug build type includes LeakCanary." — Build types don't bring libraries. You add it with `debugImplementation(…)`, so it never reaches release.

FOLLOW-UP

"A library module has no `staging` build type. What happens?" — The app's `staging` variant can't be matched. Add `matchingFallbacks += listOf("release")` to the app's `staging` build type.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Per-environment configuration

```
android {
    buildFeatures {
        buildConfig = true   // off by default since AGP 8.0
        resValues = true     // off by default since AGP 9.0
    }
    productFlavors {
        create("dev") {
            dimension = "env"
            buildConfigField(
                "String", "API_URL", "\"https://dev.example.com\"",
            )
            resValue("string", "app_name", "Shop Dev")
        }
    }
}
```

- `BuildConfig.API_URL` in code, `@string/app_name` in the manifest; with an `applicationIdSuffix`, dev and prod install side by side
- Values that must change without a release (feature switches, kill switches) belong in remote config, not in flavors — Day 24



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Where secrets do *not* go

INTERVIEW TRAP

"Keep API keys out of the code: put them in `local.properties` *and inject them via* `BuildConfig`*."* — That keeps them out of **git**, not out of the **APK**. Anything in `BuildConfig`, resources or native code can be extracted with jadx or apktool. Real secrets stay on your server; the app gets short-lived tokens — Day 12.

FOLLOW-UP

"Why is `BuildConfig` *off by default now?"* — Generating it in every module costs build time, and changing a field recompiles that module. Enable it only in the modules that read it.

FLAVOR OR RUNTIME CONFIG?

Flavors are for things that must differ **in the binary**: application ID, signing, bundled SDKs, store builds. Everything else is cheaper as runtime configuration.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22



Mid-Level

Version conflicts · BOMs & platforms · Constraints & rich versions · Lazy custom tasks



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. How Gradle picks a version

Your app declares OkHttp 4.9.3; an SDK you use needs 4.12.0. A classpath can contain only one — which wins?

- By default Gradle picks the **highest** requested version ("optimistic upgrade") — not Maven's nearest-wins
- Resolution happens **per classpath**: `debugRuntimeClasspath` and `releaseRuntimeClasspath` can differ
- Platforms (Q5), constraints and rich versions (Q6) take part in the same process

```
./gradlew :app:dependencies \
    --configuration releaseRuntimeClasspath
```

```
+--- com.example:payments-sdk:3.2.0
|    \--- com.squareup.okhttp3:okhttp:4.12.0
\--- com.squareup.okhttp3:okhttp:4.9.3 -> 4.12.0 (*)
```

`->` means conflict resolution replaced the version you asked for; `(*)` means the subtree was already shown above.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Asking Gradle *why*

```
./gradlew :app:dependencyInsight \
    --dependency okhttp \
    --configuration releaseRuntimeClasspath
```

It explains the selection reason: conflict resolution, a constraint, a platform, a strict version, a substitution.

INTERVIEW TRAP

"The version in my build file is the version that ships." — Only if nothing asks for a higher one. A transitive upgrade can bring a binary-incompatible version, and you find out through a `NoSuchMethodError` at runtime.

FOLLOW-UP

"How do you catch surprise upgrades in code review?" — Make the resolved graph part of the diff: dependency locking (`gradle.lockfile`) or a baseline tool such as Dropbox's Dependency Guard.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. BOMs and platforms

```
dependencies {
    val composeBom = platform(libs.androidx.compose.bom)
    implementation(composeBom)
    androidTestImplementation(composeBom)

    // No versions: the BOM supplies them
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui.tooling.preview)

    implementation(platform(libs.firebase.bom))
    implementation(libs.firebase.messaging)
}
```

- A BOM is a published list of versions. `platform()` imports them as **recommendations** that take part in conflict resolution like any other requested version
- Libraries released together stay aligned: all Compose artifacts, all Firebase SDKs
- In the catalog, BOM-managed entries simply omit `version`



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. What BOMs don't do

INTERVIEW TRAP

"Adding the BOM adds the libraries." — A BOM adds **no** dependencies — only versions for the ones you declare.

INTERVIEW TRAP

"An explicit version overrides the BOM." — Only if it's **higher**: the highest version wins. Going lower needs a `strictly` version — and that's a smell.

INTERVIEW TRAP

"Use `enforcedPlatform()` to be safe." — It overrides every version in the graph, including versions other libraries need. In a published library it forces your choices on every consumer. Prefer `platform()`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Constraints, rich versions and exclusions

Tool	What it does	Use it for
`constraints { }`	Sets a minimum version *if* the module appears; adds no dependency	Patching a vulnerable transitive dependency
`strictly` / `1.2.3!!`	Nothing else may change this version — it can even downgrade	Rare, documented escape hatches
`reject(…)`	Refuses specific versions	A known-broken release
`exclude(…)`	Removes a transitive module	Duplicate classes — once you know it's unused
Dependency substitution	Replaces a module with another module or a local project	Forks, composite builds

INTERVIEW TRAP

"`exclude` *fixes duplicate classes.*" — It fixes the build error and can create a `ClassNotFoundException` at runtime if code still needs that module. Check who uses it first (`dependencyInsight`).



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. In code

```
dependencies {
    constraints {
        implementation("com.example:json-parser:2.4.1") {
            because("versions before 2.4.1 have a known CVE")
        }
    }
    implementation("com.example:analytics:5.0.0") {
        exclude(group = "com.example", module = "old-logger")
    }
    implementation("com.example:image-loader") {
        version {
            require("3.1.0")
            reject("3.2.0")   // crashes on API 26
        }
    }
}
```

- `because(…)` shows up in `dependencyInsight` — the next person knows why the rule exists
- Every pin needs an owner and an exit plan; forgotten pins block upgrades for years



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Custom tasks, the lazy way

```
abstract class ReleaseNotesTask : DefaultTask() {
    @get:Input
    abstract val versionName: Property<String>

    @get:InputFile
    @get:PathSensitive(PathSensitivity.RELATIVE)
    abstract val changelog: RegularFileProperty

    @get:OutputFile
    abstract val output: RegularFileProperty

    @TaskAction
    fun generate() {
        val text = changelog.get().asFile.readText()
        output.get().asFile
            .writeText("v${versionName.get()}\n$text")
    }
}
```

- `abstract` properties: Gradle instantiates `Property`, `RegularFileProperty`, `DirectoryProperty` and `ListProperty` for you
- Declared inputs and outputs give up-to-date checks; add `@CacheableTask` when the output is worth caching



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Registering and wiring

```
val notes = tasks.register<ReleaseNotesTask>("releaseNotes") {
    versionName.set(providers.gradleProperty("appVersion"))
    changelog.set(layout.projectDirectory.file("CHANGELOG.md"))
    output.set(layout.buildDirectory.file("notes/release.txt"))
}

tasks.register<Zip>("bundleNotes") {
    // Wiring the output also wires the task dependency
    from(notes.flatMap { it.output })
    destinationDirectory.set(layout.buildDirectory.dir("dist"))
}
```

INTERVIEW TRAP

Last year's version ran a script with `project.exec {}` inside `doLast`. `Project.exec` was deprecated in Gradle 8.11 and **removed in Gradle 9**, and touching `project` at execution time breaks the configuration cache. Use the `Exec` task type, inject `ExecOperations`, or `providers.exec {}` for values.

INTERVIEW TRAP

`dependsOn("taskA")` plus a hard-coded path to its output. Wire the output `Provider` instead: the dependency is inferred, and the path can change without breaking you.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22



Senior

R8 keep rules · Measuring build performance · Remote build cache



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. R8 keep rules: where they come from

Source	Applies to
`proguard-android-optimize.txt` (from AGP)	Baseline Android rules
Your app's `proguard-rules.pro`	Your app
A library's `consumerProguardFiles(…)` — shipped inside the AAR	Every app that uses the library
Rules generated by AAPT2	Classes referenced from the manifest and XML layouts

```
# Fields read by reflection (e.g. Gson DTOs)
-keepclassmembers class com.shop.data.dto.** {
    <fields>;
}
# Class created by name, including its constructor
-keep class com.shop.plugins.PdfRenderer {
    <init>();
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Keep options and AGP 9 changes

Rule	Effect
`-keep`	Class and the listed members survive, names unchanged
`-keepclassmembers`	Listed members survive — *if* the class survives for other reasons
`-keepnames`	May still be removed if unused; if kept, not renamed
`,allowobfuscation`	Kept, but may be renamed

AGP 9

`-keep class A` **no longer implies** keeping its default constructor — write `-keep class A { <init>(); }`. Global options such as `-dontobfuscate` in a library's consumer rules now fail library publishing, and apps ignore them in prebuilt AARs. `getDefaultProguardFile()` accepts only `proguard-android-optimize.txt`.

INTERVIEW TRAP

`-keep class com.shop.** { *; }` "to be safe" switches off shrinking and obfuscation for your whole codebase. Keep narrowly, prefer code generation (KSP, kotlinx.serialization) over reflection, and run UI tests against the minified build.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Measuring build performance

Tool	Answers
Build scan (`--scan`)	Timeline, configuration time, cache hits and misses, dependency downloads
Android Studio **Build Analyzer**	Tasks that dominate the build, configuration-cache compatibility, Jetifier still needed?
Gradle Profiler	Repeatable benchmarks of realistic scenarios, with warm-ups
`--profile`	A simple local HTML report under `build/reports/profile`

```
# performance.scenarios
cart_change {
    tasks = [":app:assembleDebug"]
    apply-non-abi-change-to = "cart/src/main/kotlin/Cart.kt"
    warm-ups = 3
    iterations = 10
}
```

```
gradle-profiler --benchmark --project-dir . \
    --scenario-file performance.scenarios cart_change
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What to look at first

- **Configuration time** — it's paid on every build without a cache hit, and on every sync
- **Cache misses** on tasks that should be cacheable: absolute paths, timestamps, different JDKs
- **kapt**, **Jetifier** (`android.enableJetifier`), dynamic versions, GC time
- A giant `:core` module that everything depends on: each change recompiles the world

INTERVIEW TRAP

"Gradle Enterprise is the tool for this." — It was renamed **Develocity** in 2024. Build scans stay free on scans.gradle.com; Develocity adds a self-hosted server, a remote cache and team-wide analytics.

INTERVIEW TRAP

Last year's build-analytics design used `BuildListener` and `TaskExecutionListener`. Both are deprecated and don't work with the configuration cache. Use Develocity, or a shared `BuildService` implementing `OperationCompletionListener`, registered through `BuildEventsListenerRegistry`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. The remote build cache

```kotlin
// settings.gradle.kts
val isCi = providers.environmentVariable("CI").isPresent
fun env(name: String) = providers.environmentVariable(name).orNull

buildCache {
    local { isEnabled = !isCi }
    remote<HttpBuildCache> {
        url = uri("https://cache.example.com/cache/")
        isPush = isCi
        credentials {
            username = env("CACHE_USER")
            password = env("CACHE_PASSWORD")
        }
    }
}
```

- **CI writes, developers read.** A developer's machine can't poison the shared cache
- Backends: Develocity's build cache node, or any HTTP server that implements Gradle's `GET` / `PUT` protocol
- Enable caching itself with `org.gradle.caching=true` (Day 21)



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. When the remote cache doesn't help

INTERVIEW TRAP

Last year's version enabled it with `org.gradle.caching.remote.enabled=true`. That property doesn't exist: the remote cache is configured in `settings.gradle.kts` (`buildCache { remote … }`).

INTERVIEW TRAP

"We added a remote cache, so CI is fast." — Only if keys match. Absolute paths in inputs, different JDK vendors or OS line endings, and generated timestamps turn every task into a miss. A build scan shows *which input* differed between two builds.

FOLLOW-UP

"Can a remote cache make builds slower?" — Yes. Downloading an output has a cost; for cheap tasks over a slow network, rebuilding locally wins. Measure hit rate **and** time saved.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22



Architect

AGP upgrade strategy · Dependency governance · Variant API at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. An AGP upgrade strategy

Piece	Couples with
Android Studio	Each version supports a range of AGP versions
AGP	A minimum Gradle version; JDK 17+ to run (AGP 9.0 needs Gradle 9.1+)
Kotlin Gradle plugin	Tested AGP and Gradle ranges; AGP 9 depends on KGP at runtime
KSP, Hilt, other plugins	Must support your AGP, Kotlin and Gradle versions

1. Read the release notes and the compatibility tables **first**
2. Fix deprecation warnings on the **current** version (`--warning-mode all`) — the next major removes those APIs
3. Run the **AGP Upgrade Assistant**; upgrade one tool per PR
4. Build previews on a branch (Android Studio canary, AGP alpha/beta) to find breakage early
5. Verify: build-scan comparison, APK/AAB diff (size, manifest, DEX), release UI tests on the minified build



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. AGP 9 in particular

AGP 9

- **Built-in Kotlin** is on by default: remove `org.jetbrains.kotlin.android`; `kotlin-kapt` must move to KSP or `com.android.legacy-kapt`
- **New DSL only**: the old `BaseExtension`-style types and the legacy variant API (`applicationVariants`) are hidden
- Opt-outs (`android.newDsl=false`, `android.builtInKotlin=false`) are temporary bridges — they're planned to go away in AGP 10

INTERVIEW TRAP

"We'll stay on the old AGP until something forces us." — Play's targetSdk requirement rises every year, a new compileSdk needs a new AGP, and skipping several majors turns a routine upgrade into a project. Small, regular upgrades are cheaper.

INTERVIEW TRAP

"Opt-out flags make AGP 9 done." — They only postpone the migration. Track every third-party plugin that still needs them and plan their removal.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Dependency governance at scale

Problem	Control
Versions drift between modules and repositories	One catalog + an internal **platform** (BOM) that every app imports
Misdeclared or unused dependencies	Dependency Analysis Gradle Plugin (`buildHealth`) in CI
Surprise upgrades	Dependency locking or a dependency-diff check on each PR
Supply-chain attacks, dependency confusion	Dependency verification; `exclusiveContent` for internal groups
Stale libraries	Renovate/Dependabot with small, frequent PRs

INTERVIEW TRAP

"Adding a library is a one-line change." — It adds its whole transitive graph: APK size, keep rules, licenses, and possibly silent upgrades of versions you already use. New third-party dependencies deserve a review.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. An internal platform and repository filtering

```
// platform/build.gradle.kts — the company BOM
plugins { `java-platform` }
javaPlatform { allowDependencies() }

dependencies {
    api(platform(libs.androidx.compose.bom))
    constraints {
        api(libs.okhttp)
        api(libs.kotlinx.coroutines.core)
    }
}
```

```
// settings.gradle.kts — internal groups only from our repo
dependencyResolutionManagement.repositories {
    exclusiveContent {
        forRepository { maven("https://maven.example.com") }
        filter { includeGroupByRegex("com\\.shop(\\..*)?") }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Variant explosion and the Variant API

Two dimensions of three flavors and three build types is already 27 variants. Each one adds configuration time, sync time and CI surface.

```
androidComponents {
    beforeVariants { variant ->
        // Nobody ships or tests the dev backend in release
        if (variant.buildType == "release" &&
            variant.productFlavors.contains("env" to "dev")
        ) {
            variant.enable = false
        }
    }
}
```

- Disabled variants never get tasks — cheaper than ignoring them
- Prefer runtime configuration over new flavor dimensions; for many white-label brands, prefer app modules sharing feature modules



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Custom tasks per variant

```
androidComponents {
    onVariants(selector().withBuildType("release")) { variant ->
        tasks.register<ApkSizeTask>("${variant.name}ApkSize") {
            apkDir.set(variant.artifacts.get(SingleArtifact.APK))
            loader.set(variant.artifacts.getBuiltArtifactsLoader())
        }
    }
}
```

- `SingleArtifact.APK` is a lazy `Provider`: wiring it adds the dependency on the packaging task automatically
- Inside the task: `@get:InputFiles abstract val apkDir: DirectoryProperty` and an `@Internal BuiltArtifactsLoader` to list the APKs

INTERVIEW TRAP

`applicationVariants.all { }` plus `dependsOn("assembleRelease")` and a hard-coded `build/outputs/apk/…` path. The legacy variant API is disabled by default in AGP 9, and output paths are an implementation detail. Use `androidComponents` and the artifacts API.

SENIOR ANSWER IN ONE LINE

"Declare narrowly, inspect the resolved graph, wire outputs instead of paths — and upgrade in small, measured steps."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 22 · ARCHITECT

Day 22 — cheat sheet

- `implementation` by default; `api` only for types in your public API. Both end up in the APK.
- Variants = one flavor per dimension × build type. Flavors for binary differences, runtime config for the rest.
- R8 is **off** unless `isMinifyEnabled = true`. Test the minified build.
- `BuildConfig`/resources are not secret storage. `buildConfig` (AGP 8) and `resValues` (AGP 9) are off by default.
- Gradle picks the **highest** requested version. Check `releaseRuntimeClasspath` with `dependencies` / `dependencyInsight`.
- BOMs add versions, not libraries. Prefer `platform()` over `enforcedPlatform()`; every pin needs a `because()`.
- Custom tasks: abstract `Property` inputs, annotated outputs, `register`, wire `Provider`s. No `project` in task actions.
- Keep rules: narrow, reflection-driven. AGP 9: `-keep class A` no longer keeps the constructor.
- Measure with build scans and Gradle Profiler; remote cache: CI pushes, developers pull.
- Upgrades: fix deprecations first, one tool per PR. AGP 9 = built-in Kotlin + new DSL + `androidComponents`.

Day 23 is next → CI/CD, part 1: pipelines, caching & static analysis.
Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700